Build System Pipeline

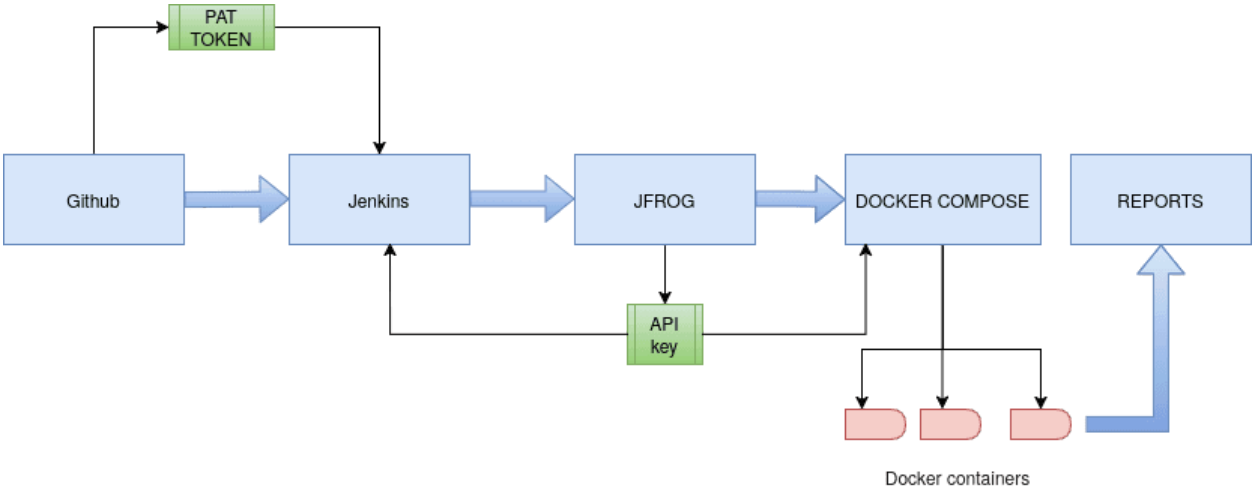
Objective:

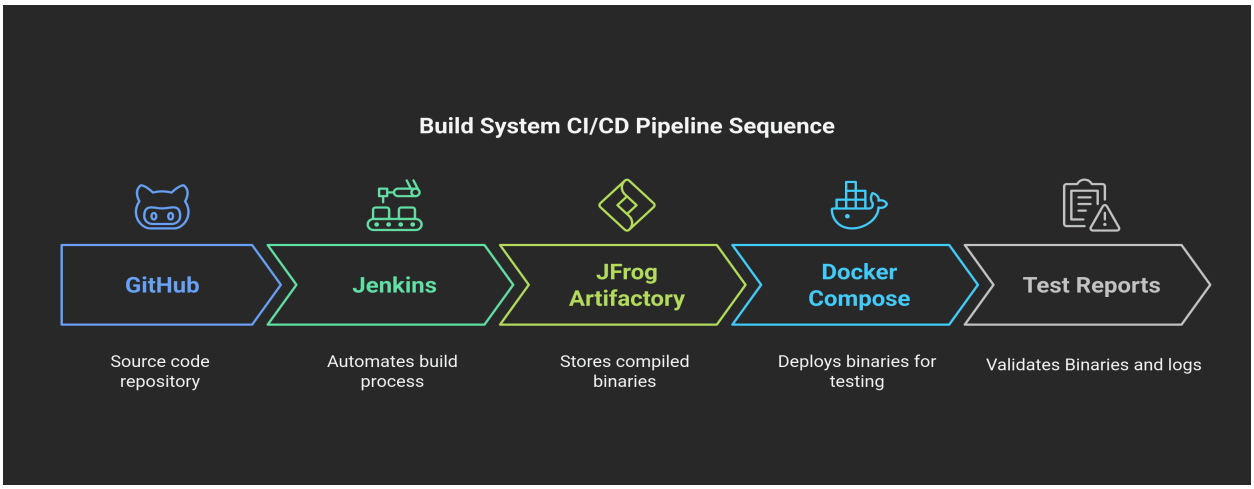
The goal of this pipeline is to **compile a package from source**, test it on a local system, and generate a .tar.gz or .deb package. If required, the final package will be uploaded to internal repositories for wider distribution.

Main Components

- **Source Code Management (SCM)**:
  GitHub – Manages and stores source code.
- **Continuous Integration (CI)**:
  Jenkins – Automates build, packaging, and testing.
- **Artifact Repository**:
  JFrog Artifactory – Stores packaged binaries (.tar.gz, .deb).
- **Deployment**:
  Docker Compose – Manages multi-container environments.
- **Testing & Validation**:
  Automated scripts ensure package **integrity** and **correctness**.

Basic Workflow





1. A Jenkins build is triggered (manual or automated).
2. Jenkins pulls source code and build scripts from cm-server.
3. Compilation script transforms the source into binary (.tar.gz/.deb).
4. The binary package is uploaded to JFrog Artifactory via secure API.
5. Docker Compose pulls the package and spins up containers.
6. Installation and testing are executed inside isolated containers.
7. Test results and logs are stored and pushed back to Jenkins console for review.

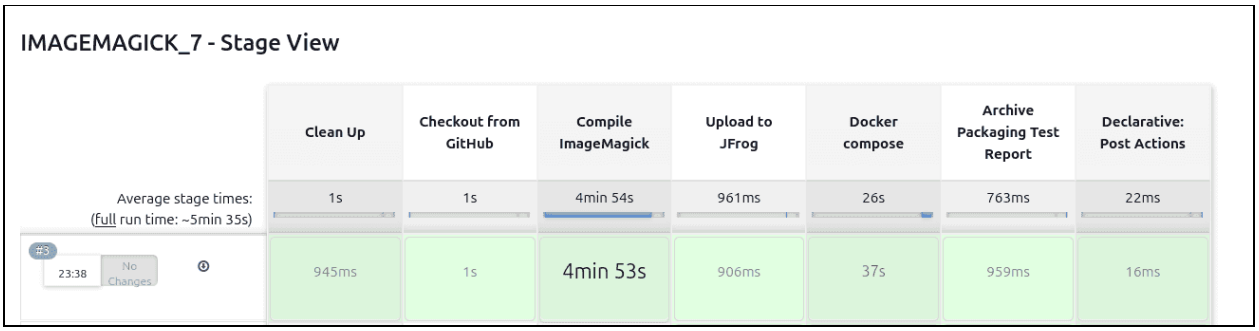
GitHub Repository Structure

```
Imagemagick_7/
├── Jenkinsfile
├── package_compilation_script.sh
├── docker-compose.yaml
├── Dockerfile
├── test_imagemagick.sh
└── README.md
```

- **Jenkinsfile**: Pipeline-as-Code definition for Jenkins CI.
- **package_compilation_script.sh**: Contains the logic for source compilation and packaging.
- **docker-compose.yaml**: Manages multi-container setup for testing.
- **Dockerfile**: Defines the custom image used to spin up containers.
- **test_imagemagick.sh**: Sample test script to verify ImageMagick installation.

Jenkins Pipeline Stages

- **Cleanup Stage** – Ensures workspace is fresh and clean.
- **Repo Clone Stage** – Clones the GitHub repository.
- **Compilation Stage** – Compiles source and generates the .tar.gz package.
- **Artifact Upload** – Securely uploads the binary to JFrog.
- **Docker Compose Execution** – Spins up containers for testing.
- **Testing & Report Collection** – Executes test scripts and stores the output.



JFrog Artifactory

- Centralized **binary repository**.
- Serves as an **internal package hub**.
- Supports versioning and controlled distribution across environments.

Docker Compose & Containerization

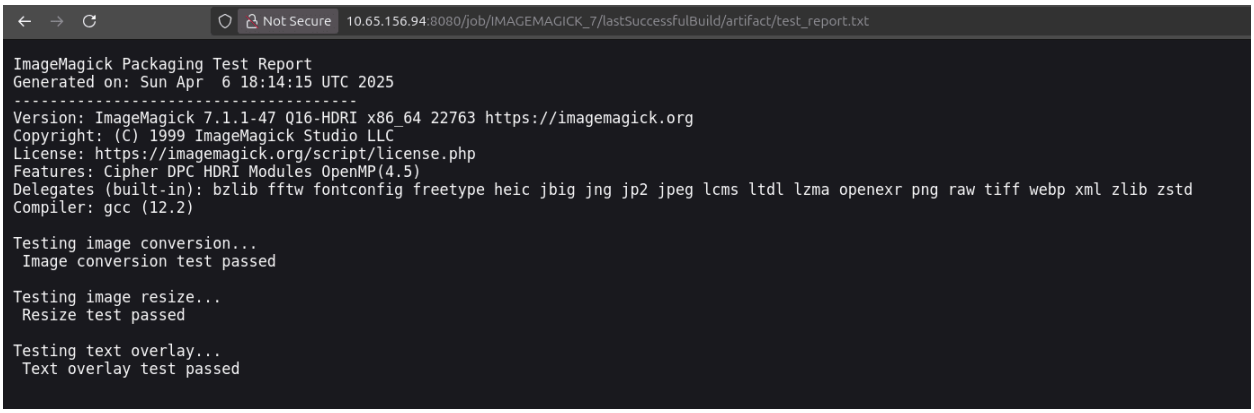
- Docker Compose is used to orchestrate multiple containers.
- The **YAML manifest** ensures environment consistency.
- Uses the **Dockerfile** to build lightweight custom images.
- After test execution, containers are automatically removed for stateless testing.

Dockerfile

Used to create a **custom Docker image** containing minimal utilities required for testing the package. The installer and test scripts are executed inside this container and removed afterward.

Testing & Reports

- The test report validates whether the compiled package **functions correctly** inside a clean container environment.
- Example: For ImageMagick, convert -version is validated through test_imagemagick.sh.
- **Volume Mount** is used to persist reports/logs outside the container (host_path:container_path).
- Final test reports are:
  - Available in Jenkins Web UI
  - Shareable with application teams (email/reporting tool)
  - Used as a QA gate before pushing to production



Current Test Setup

- **Package**: ImageMagick
- **SCM**: GitHub
- **Jenkins**: http://10.65.156.94:8080
- **JFrog**: http://10.65.150.52:8082
- **Docker Host**: 10.65.149.232

Future Scalability

Currently, we handle compilation of up to **5–10 packages**:

1. Node.js
2. ImageMagick
3. Docker
4. Python
5. FFmpeg

In the short term, Jenkins **master and runner** are on the same node for simplicity.
In the long term, if the number of packages increases, we will **adopt a master-agent model** where Jenkins runners are offloaded to separate nodes for scalability and efficiency.

Key Considerations

As we evolve this pipeline, the following architectural concerns will be analyzed and optimized:

1. **Security**
2. **Reliability**
3. **Resource Usage**
4. **Reusability**
5. **Environment Fit**
6. **Drawbacks & Bottlenecks**

Security Best Practices

Security is **core** to this pipeline. We've implemented:

- **Personal Access Tokens (PATs)**
  For secure communication between GitHub and Jenkins.

- **API Keys**
  Used securely to interact with JFrog for pushing/pulling artifacts.
- **Jenkins Credentials Management**
  Sensitive credentials (PAT/API Keys) are stored **securely** using Jenkins' native credential store.
    - These values are masked in console output.
    - Parameters are securely injected into the build at runtime.